Sentinel
Team Task Plan

Ownership, tasks and timeline
for the eight-member team

Graduation Project | Autonomous Security Operations Platform | October 2026

Contents

1. Team Organisation Overview 3
2. Habiba - Autonomous AI Agent and ML Inference 4
3. Alaa - Data Pipeline, Detection and Correlation 6
4. Mohamed Osama - SOAR Core: Playbooks, Executor, Response and Cases 7
5. Omar - Penetration-Testing Module and Attack Simulation 8
6. Ariam - Threat Intelligence and MITRE ATT&CK; 9
7. Mostafa - Dashboard Platform and Operations Pages 10
8. Rawda - GRC Part One: Assets, Risks and Test Data 11
9. Amira - GRC Part Two: Compliance, Evidence, Policies, Reports and Notifications 12
10. Shared Contracts (Agree in Week 1) 13
11. Review Pairs, Timeline and Milestones 14

1. Team Organisation Overview

1.1 Goal

Deliver the Sentinel platform (see the Project Details document) in about 12 weeks with eight people working in parallel. The work is divided into **eight ownership areas**. Each person owns one area end to end: the backend component, its data, its tests, and **its own dashboard pages**. Owning the full vertical slice avoids hand-off delays and makes it clear who answers questions about each part.

1.2 Starting point

Already complete	Owner
ML attack classifier: trained Random Forest, scaler, feature list, evaluation report	Habiba
Customer website (skyline-web) and platform API	Omar, Mostafa, Mohamed Osama

1.3 Ownership map

Member	Ownership area	Main output
Habiba	Autonomous AI Agent and ML inference	The decision-making brain of the platform
Alaa	Data pipeline, detection and correlation	Events in, incidents out
Mohamed Osama	SOAR core: playbooks, executor, response, cases	Everything that acts automatically
Omar	Penetration-testing module and attack simulation	Findings, detection tests, before/after proof
Ariam	Threat intelligence and MITRE ATT&CK	Evidence for every decision
Mostafa	Dashboard platform and operations pages	The main interface
Rawda	GRC part one: assets, risks, test data	Business risk view and demo data
Amira	GRC part two: compliance, evidence, policies, reports, notifications	Governance and reporting

1.4 Working agreements

- **One repository**, one folder per component (`pipeline/`, `agent/`, `executor/`, `pentest/`, `intel/`, `dashboard/`, `grc/`, `common/`). Each component has its own Dockerfile and README.
- **Branches and reviews**: feature branches, pull requests, and at least one reviewer from the review table in section 11. Nothing goes to `main` without a passing test run.
- **Contracts first**: the shared event schema, stream names and API shapes (section 10) are agreed in week 1 and changed only through a pull request that all affected owners approve.
- **Weekly integration session** (60 minutes): each owner demonstrates their component running inside the shared Docker Compose stack.
- **Definition of done** for every task: code merged, tests written and passing, README updated, a short demo or screenshot, and (where it has a UI) the dashboard page working with real data.
- **Mock first**: until a neighbouring component exists, develop against a mock that follows the contract, so nobody waits.

2. Habiba - Autonomous AI Agent and ML Inference

Mission: build the agent that replaces the Tier-1 analyst: it reads an incident, gathers evidence with tools, decides, and triggers automatic action inside strict safety limits. Also package the finished ML model as a production service.

ID	Task	What to do	Done when	Needs / weeks
H1	LLM gateway	One module for all model calls: provider selection, timeouts, retries, fallback model, token and cost counters, request logging with correlation id	Any agent can call the model through one function; usage per call is stored	None / 1
H2	Tool registry and tool-calling loop	Define each tool (name, description, JSON schema, permission scope, validator); implement the bounded reasoning loop (max steps, max seconds); store every tool call	Agent can run a read-only investigation on a sample incident and return a decision	H1 / 2
H3	Context builder	Code that assembles the compact incident JSON (alerts, evidence, asset, history, policies), masks personal data, truncates and delimits attacker-controlled strings	Output is deterministic, size-limited and contains no raw secrets	Alaa schema, Ariam evidence / 1
H4	Prompt library	System prompts for Triage, Investigator, Responder and Reporter; three or more few-shot examples per attack family including a false positive and an uncertain case; versioning	Prompts stored with versions; each decision records the version	H2 / 2
H5	Decision schema and validator	Pydantic model for the verdict, severity, confidence, evidence, techniques, actions and reasoning; retry once on invalid output	Invalid model output never reaches the executor	H4 / 1
H6	Confidence gates and Policy Guard integration	Implement the gate table (G0 to G3); send proposed actions through Mohamed Osama's Policy Guard; handle rejections	An action above its gate is rejected and logged	M4 / 1
H7	Memory	Per-IP profile (first seen, counts, last verdicts and actions); store analyst corrections and feed them back as examples	Second incident from the same IP shows history in the context	H3 / 1
H8	Fallback decision engine	Deterministic rules producing the same schema when the model is unavailable or invalid	Turning the model off keeps the pipeline working; decisions marked fallback	H5 / 1
H9	Agent defences	Prompt-injection test suite (crafted user-agents, URLs, form fields), tool-permission tests, rate limits per tool	Test suite passes; no forbidden action under any test input	H6 / 1
H10	Evaluation harness	Build a golden set of 50 or more incidents with correct verdicts and actions; run agent versus fallback; report accuracy, wrong-action rate, latency, cost	A report file generated by one command	H8 / 1.5
H11	Reporter and Tuning agents	Reporter writes incident summary and lessons learned; Tuning agent proposes thresholds or allowlist changes from false-positive feedback	Summaries appear in cases; proposals appear as records	H5 / 1.5

ID	Task	What to do	Done when	Needs / weeks
H12	ML inference service	Wrap the trained model, scaler and feature list in a prediction API returning label, probabilities and top features; unit test that live sniffer features match training columns; per-class evaluation report	Detector (Alaa) calls the API and gets valid predictions	Alaa / 1
H13	Model drift monitor	Daily statistics of predicted classes and key features; alert on large shifts	Chart on the AI Agent page	H12 / 0.5
H14	Dashboard: AI Agent Console	Decision log, reasoning view, confidence histogram, fallback rate, cost per day, autonomy mode switch (Autonomous, Shadow, Paused)	Page shows live decisions from the running system	Mostafa framework / 1.5

3. Alaa - Data Pipeline, Detection and Correlation

Mission: turn raw network and application activity into clean, ordered events, detect attacks in them, and group the detections into incidents. Everything downstream depends on this area.

ID	Task	What to do	Done when	Needs / weeks
A1	Unified event schema	Define the Event model (fields in the Project Details document), publish it as a shared package in `common/`	All components import the same model	None / 0.5
A2	Redis Streams backbone	Create streams (events, alerts, incidents, actions, audit), consumer groups, capped lengths, acknowledgement, dead-letter stream, helper library for producers and consumers	A test producer and two consumers show no loss when one consumer is killed	A1 / 1.5
A3	Sniffer to stream	Change the cicflowmeter capture so each flow is written to the events stream instead of the CSV file; confirm feature names and order	Flows appear in the stream within seconds	A2 / 1
A4	Log shipper	Tail skyline-web and HAProxy logs, parse into events; add the website auth events; enable `option forwardfor` in HAProxy so the real client address is logged	Failed logins and requests appear as events with the correct client IP	A2, website team / 1.5
A5	Detector service	Consume events, call the ML service, apply the rate layer; **fix the existing** `flows_in_window` **KeyError**; write alerts to the alerts stream and database	No crashes under test load; every alert has engine, confidence and evidence	H12 / 2
A6	Rule engine	YAML rules (single, threshold, sequence), test events per rule, loader with validation; ship at least 15 rules (brute force, credential stuffing, scanning, SQL injection patterns, upload checks, abnormal status codes)	Each rule passes its test; rules reload without restart	A5 / 2
A7	Correlation engine	Merge alerts into incidents by IP and attack family in sliding windows; cross-source corroboration; sequence detection; close after quiet period	Fifty alerts from one flood become one incident	A5 / 2
A8	Severity scoring	Implement the weighted formula with configuration file; store the component values	Dashboard can display the score breakdown	A7, Rawda assets, Ariam reputation / 1
A9	Stick-table protection and detection metrics	Configure HAProxy stick tables for flood rejection; metrics for detection latency	Flood test shows 429 responses before any agent decision	A4 / 1
A10	Dashboard: Sources and Pipeline Health	Events per second per source, last-seen, stream lengths, lag, dead-letter count	Page goes red when a source is silent	Mostafa framework / 1
A11	Master Docker Compose and CI	One compose file with every service, health checks, `.env` template, and a CI job that runs unit tests	`docker compose up` starts the whole platform	All owners / 1 (ongoing)

4. Mohamed Osama - SOAR Core: Playbooks, Executor, Response and Cases

Mission: build everything that carries out decisions automatically and safely: the playbook engine, the executor, the HAProxy integration, the safety layer, and case management.

ID	Task	What to do	Done when	Needs / weeks
M1	Playbook engine	YAML interpreter: triggers, steps, conditions, parallel branches, variables, retries with backoff, timeouts, dry-run mode, run history stored per step	A sample playbook runs end to end and shows step-by-step output	A2 / 2.5
M2	Executor service	Action workers reading the actions stream; idempotency keys; retry and error handling; structured results; audit records for every action	The same action sent twice is executed once	A2 / 2
M3	HAProxy connector	Enable the runtime socket; ACL blocklist file; commands `add acl`, `del acl`, `show acl`; file and live-ACL reconciliation; TTL scheduler; verification step; rate-limit and tarpit helpers	An IP is blocked and unblocked at expiry without restarting HAProxy; verification recorded	A4 / 2
M4	Policy Guard	Code that validates every proposed action: allowlist, maximum duration per gate, maximum active blocks, maximum blocks per minute (circuit breaker), scope check against assets	A blocked-IP-in-allowlist attempt is rejected and logged	None / 1.5
M5	Kill switch and autonomy modes	Global and per-playbook switch honoured by every worker; Autonomous, Shadow and Paused modes	Switching to Paused stops actions within seconds	M2 / 1
M6	Website hooks	Add authenticated platform endpoints on the website: lock or flag an account, revoke sessions, add a request-pattern block; connectors in the executor	Actions work against website mode A and B	Website team / 1.5
M7	Playbook library	Write the ten playbooks from the Project Details document and their tests	Each playbook passes dry-run and a live scenario	M1 / 2
M8	Case management backend	Case creation rules, states, SLA timers, notes, timeline entries, closure form; API endpoints	An incident above Low creates a case automatically	Alaa incidents / 2
M9	Dashboard: Playbook Studio	List, YAML editor with validation, dry-run button, run history, version rollback	Edit and test a playbook from the UI	M1, Mostafa framework / 1.5
M10	Dashboard: Autonomous Actions Ledger and Response page	Every action with result and verification; Undo and Extend; active blocks, allowlist management; HAProxy sync status	Undo removes the block in HAProxy	M3 / 1.5

5. Omar - Penetration-Testing Module and Attack Simulation

Mission: make the customer testable and the platform provable. Build the two website versions, the controlled scanning pipeline, and the attack scenarios used to validate detection and for the final demonstration.

ID	Task	What to do	Done when	Needs / weeks
O1	Website mode A (vulnerable)	Add deliberate, documented weaknesses behind `MODE=vulnerable`: string-built SQL in search, unescaped comments, weak password storage, unrestricted upload, missing headers, no rate limit, verbose errors	A list of intended weaknesses exists and each can be demonstrated	None / 1.5
O2	Website mode B (hardened)	Fix each weakness behind `MODE=secure`: parameterised queries, output escaping, strong hashing, upload validation, rate limiting, headers, CSRF	Every O1 weakness has a matching fix and a test	O1 / 1.5
O3	Scope lock and runner	`scope.yaml` with the allowed target; code that refuses any other target; each tool in its own container with CPU, time and network limits	A scan against a non-listed address is rejected by code	None / 1.5
O4	Tool wrappers	Wrappers for Nmap, OWASP ZAP, Nikto, sqlmap and Hydra (small wordlist) with scan profiles (quick, standard, full)	One command runs a profile and stores raw output	O3 / 2
O5	Parsers and findings model	Convert tool outputs into one finding format: title, endpoint, evidence, CWE, OWASP category, CVSS, status; store in `findings`	Findings from every tool appear in one table	O4 / 2
O6	Pentest planner	Agent that chooses which scans to run from the site map and scope, using Habiba's gateway; summarises results	A scan plan is produced and executed automatically	H1, O4 / 1.5
O7	Pentest report and retest	PDF report with executive summary, technical details, reproduction steps and fixes; compare mode A and mode B scans	Before/after table generated by one command	O5 / 1.5
O8	Pentest integration with SOAR	Marker events at scan start and end; label incidents `authorized_test`; do not block the runner; record which launched attacks were detected and how fast	Detection test table filled automatically	Alaa, Mohamed / 1
O9	Attacker container and scenarios	Separate attacker container with its own address; scripted scenarios: brute force, flood, SQL injection attempts, scanning, upload, and a legitimate heavy-use false-positive case	Each scenario can be started with one command	O1 / 1.5
O10	Dashboard: Pentest Center and Findings	Start scans, live progress, history, findings table and detail, before/after view, detection-test table	Full scan runnable from the UI	Mostafa framework / 2

6. Ariam - Threat Intelligence and MITRE ATT&CK

Mission: supply the evidence every decision depends on. Build the enrichment service, the VirusTotal and other intelligence integrations, and the MITRE ATT&CK knowledge layer.

ID	Task	What to do	Done when	Needs / weeks
R1	VirusTotal client	Calls to the IP, domain, URL and file-hash endpoints with the API key in the request header; extract malicious/suspicious counts, reputation, country, ASN, tags; handle errors	A lookup returns a clean summary object	None / 1
R2	Quota governor and cache	Token-bucket limiter (about 4 per minute and 500 per day on the public key), priority queue by severity, cache with TTL in the database, never send private or lab addresses	Hundreds of repeated lookups cause no extra API calls and no quota errors	R1 / 1.5
R3	AbuseIPDB and GeoIP	Abuse confidence lookup with cache; local GeoLite2 database for country, city, ASN and coordinates	Evidence includes location and abuse score	None / 1
R4	Enrichment service	Parallel lookups, timeouts, partial results, one evidence bundle per incident; reputation score for the severity formula	Every incident gets a bundle within seconds, even if one source fails	R2, R3 / 2
R5	MITRE ATT&CK loader	Download and load the Enterprise STIX 2.1 data (v19), index techniques, tactics, mitigations and detections; scheduled refresh; **read tactics from the data**, because Defense Evasion was split into Stealth and Defense Impairment	Lookup by technique id returns name, tactic and mitigations	None / 1.5
R6	Attack-to-technique mapping	Data file mapping attack types and rule ids to technique ids, reviewed against the loaded data; API to resolve	Every alert and incident carries technique ids	R5 / 1
R7	Coverage analysis	Compute which techniques have working detections, fired, or are uncovered; export Navigator layer JSON per incident and per period	Layer file opens in ATT&CK Navigator	R6 / 1.5
R8	Indicator store	Normalise, deduplicate, score, expire indicators; STIX export	Indicator table with freshness and export	R4 / 1.5
R9	Dashboard: Threat Intelligence Hub and ATT&CK Coverage	Lookup page with side-by-side sources, quota meters, cache rate; full matrix coloured by coverage; technique detail	Matrix updates after new detections	Mostafa framework / 2

7. Mostafa - Dashboard Platform and Operations Pages

Mission: build the main interface that everyone else plugs into: the framework, live updates, authentication and the core operations pages.

ID	Task	What to do	Done when	Needs / weeks
S1	Framework decision and skeleton	Decide React or Streamlit multi-page in week 1; layout, left navigation by role, dark and light themes, severity colour tokens, shared components (tables with server-side paging, charts, drawers)	Other owners can add a page by copying a template	None / 1.5
S2	Authentication and roles	Login, JWT, roles (Admin, Analyst, Auditor, Client), optional TOTP, route protection, role-based menu	Client role sees only the portal	S1 / 1.5
S3	Live updates	Server-Sent Events or WebSocket channel from the API for alerts, incidents, actions; connection indicator	New alert appears on screen within 3 seconds without refresh	A2 / 1.5
S4	Dashboard API	Endpoints for alerts, incidents, stats, filters, pagination; indexes; the `stats_hourly` aggregation job	Pages load in under a second with 100,000 alerts	A2 / 2
S5	Command Center	KPI tiles, alerts per hour, attack types, top IPs, top endpoints, engine contribution, live feed, SLA-at-risk list	Home page fully populated from real data	S4 / 2
S6	Alerts Explorer	Filters, saved views, bulk actions, detail drawer with raw event and model probabilities	Filtering 100,000 alerts is smooth	S4 / 1.5
S7	Incident Workbench	Header with score breakdown, timeline, evidence, entity graph, reasoning, actions, notes, reverse and escalate buttons	One incident shows the full story	S4, H14 data / 2.5
S8	Case Board	Kanban by status with drag and drop, SLA badges, case page with notes and closure form	Moving a card updates the case	Mohamed M8 / 1.5
S9	Live Threat Map	World map with attack arcs, filters, side panel	Map animates with new incidents	Ariam geo data / 1.5
S10	Admin and RBAC page	Users, roles, API keys, thresholds, notification routes, configuration history	Changing a threshold takes effect without restart	S2 / 1
S11	Global search and platform health page	Search by IP, incident, case, asset, CVE, technique; service status and quota widgets	Search jumps to the right record	S4 / 1.5

8. Rawda - GRC Part One: Assets, Risks and Test Data

Mission: give the platform its business context: what the customer owns, how much it matters, and how risky it is. Also create the data generator used for demos and load tests.

ID	Task	What to do	Done when	Needs / weeks
W1	Asset inventory model and API	Collection and endpoints for assets: name, type, owner, business service, criticality 1 to 5, data classification, exposure, environment, last seen; seed the real Skyline assets	Asset lookup by IP is available to the severity engine and the agent	A1 / 1.5
W2	Asset relationships	Link assets to incidents, findings, risks and controls; per-asset summary endpoint	Asset page can show everything related	W1 / 1
W3	Risk register model and API	Fields from the Project Details document; scoring (likelihood x impact, 1 to 25, four bands); treatment options; acceptance with expiry	A risk is created, scored and listed	W1 / 1.5
W4	Live risk calculation	Job that recalculates likelihood from incidents and open findings per asset, and impact from criticality and data class; history stored for trend charts	A new attack or finding changes the matching risk score	W3, Omar findings / 2
W5	Key Risk Indicators	Calculate the KRIs (open critical findings, remediation time, ageing, autonomy rate, coverage, expired exceptions)	KRI endpoint feeds the dashboards	W4 / 1
W6	Dashboard: Asset Inventory	Table, relationship view, per-asset panel	Click an asset to see incidents, findings, risks	Mostafa framework / 1.5
W7	Dashboard: Risk Register and Heatmap	5x5 heatmap, register table, risk detail, trend chart, acceptance workflow	Heatmap moves when risks change	W4 / 2
W8	Dashboard: Client Portal	Plain-language posture score, top risks, incident summary, compliance percentage, report downloads, list of automatic actions taken	Client role sees only this page	W5, Amira / 1.5
W9	Synthetic data and load generator	Scripts that write realistic events into the events stream (varied IPs, ports, times, attack types) at configurable rates; documented as a stress test	Ten thousand events processed end to end; metrics collected	A2 / 1.5
W10	Test fixtures	Sample incidents and evidence files for unit tests of all components	Fixtures folder used by other owners' tests	A1 / 1

9. Amira - GRC Part Two: Compliance, Evidence, Policies, Reports and Notifications

Mission: turn the platform's activity into governance: controls and compliance scores, evidence, policies that execute, executive reports, and the notification engine.

ID	Task	What to do	Done when	Needs / weeks
Z1	Control library	Data files for NIST CSF 2.0, selected ISO/IEC 27001:2022 Annex A controls, CIS v8 safeguards and OWASP Top 10 categories; each control with description, owner, status, maturity, review date; **verify numbers against the official text**	Library loads and lists at least 80 controls with mapping to platform features	None / 2
Z2	Compliance scoring	Implement the weighted formula per framework and per function; gap and quick-win lists	Percentages computed and stored daily	Z1 / 1.5
Z3	Automatic evidence collector	Jobs that gather evidence: active rules, executed actions, scan reports, audit logs, configuration snapshots; hash, timestamp, source, linked controls; stale detection	Controls show evidence counts automatically	Z1 / 2
Z4	Policy library	Policies with version, owner, approval, review date, acknowledgements; link a policy to the playbook parameters it drives; exceptions with expiry	Changing a policy value changes playbook behaviour	Mohamed M1 / 2
Z5	Remediation tracker	Tasks created from findings and control gaps: owner, due date, status; closes when retest confirms fix; overdue tasks raise risk	Task closes automatically after a successful retest	Omar O5, Rawda W3 / 1.5
Z6	Notification engine	Telegram, email and webhook senders; templates with evidence summary; severity and asset routing; deduplication; delivery tracking; escalation timers; optional chat commands	An autonomous action triggers a message that says what was done, why and how to undo	A2 / 2
Z7	Report generator	Executive report (one to two pages) and technical report as PDF, on demand and weekly; charts from stored statistics	One command produces both reports from live data	Z2, Rawda W5 / 2
Z8	Dashboard: Compliance Center and Policy Library	Gauges per framework, control table, gaps and quick wins, policy list with linked playbooks and exceptions	Page updates when evidence arrives	Mostafa framework / 2
Z9	Dashboard: Evidence and Audit and Reports pages	Evidence browser, audit log viewer (search and export), report generation and history	Auditor role can use these pages read-only	Z3, Z7 / 1.5
Z10	Documentation and diagrams	Architecture diagram, data-flow diagram, README per component, final report chapters on GRC and operations	Documentation reviewed by all owners	All / ongoing

10. Shared Contracts (Agree in Week 1)

Contract	Producer	Consumers	What is agreed
Event schema	Alaa	Everyone	Field names, types, source values, event types
Streams and consumer groups	Alaa	Detector, correlator, executor, audit	Names, message format, acknowledgement and dead-letter rules
Alert and incident records	Alaa	Agent, dashboard, GRC	Field list, severity values, status values
Evidence bundle	Ariam	Agent, severity engine, dashboard	Keys for reputation, geo, techniques, history
Asset lookup	Rawda	Severity engine, agent, risk job	Lookup by IP or name, criticality scale
Decision record	Habiba	Executor, cases, dashboard	Schema in section 2 of this plan and in the Project Details document
Action request and result	Mohamed Osama	Agent, dashboard, audit	Tool names, arguments, result and verification fields
Finding record	Omar	GRC, agent, dashboard	Title, endpoint, CWE, severity, status, evidence
Audit record	Everyone	Audit writer	Actor, action, target, result, timestamp, correlation id
Dashboard page template	Mostafa	Everyone	Layout, theme tokens, table and chart components

11. Review Pairs, Timeline and Milestones

11.1 Code review pairs

Author	Reviewer	Reason
Habiba	Alaa	Agent consumes the pipeline; both understand the data
Alaa	Mohamed Osama	Detection output drives the executor
Mohamed Osama	Habiba	Safety layer must match the agent's gates
Omar	Mostafa	Both worked on the website and API
Ariam	Habiba	Evidence feeds the agent context
Mostafa	Omar	Dashboard and website share the API style
Rawda	Amira	Both own GRC data models
Amira	Ariam	Reports consume intelligence and coverage data

11.2 Timeline

Weeks	Focus	Milestone
1-2	Contracts, schema, streams, framework skeleton, website modes, mock services	**M1**: events flow from sniffer and logs to the alerts stream; dashboard skeleton running
3-5	Detection, correlation, enrichment, agent loop, executor, HAProxy blocking, asset and risk models	**M2**: first full chain: attack, alert, incident, enrichment, agent decision, block
6-8	Cases, pentest pipeline, control library, evidence, notifications, dashboard pages	**M3**: all pages show live data; pentest findings feed risks
9-10	Reports, retest, attack scenarios, golden set, injection tests, load test, chaos tests	**M4**: acceptance criteria measured and recorded
11	Integration fixes, documentation, demo rehearsal	**M5**: full demonstration script runs without errors
12	Buffer for delays and polish	**M6**: final delivery

11.3 If time becomes short

Protect this chain first: pipeline, correlation, enrichment, agent with fallback, executor with HAProxy blocking, cases, Command Center, Incident Workbench, a basic pentest run with before/after, and a simplified GRC view (assets, risk heatmap, compliance gauges). Optional extras from the Project Details document come last.

11.4 Demonstration roles

Every owner presents their own component in the final demonstration: Omar launches the scenarios, Alaa shows the pipeline and incident, Ariam shows the evidence, Habiba shows the agent reasoning, Mohamed Osama shows the automatic block and undo, Mostafa walks through the dashboard, Rawda and Amira show risk, compliance and the reports.